



جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

BANKING DATABASE MANAGEMENT SYSTEM

CSE371 Database Project

BY

MOURAD ELSHORBAGY	22-101026
RANA WAEL	22-101219
AMR EDRIS	22-101097
ARWA MOSTAFA	22-101120
ETHAR MOSTAFA	22-101121

EGYPT UNIVERSITY OF INFORMATICS

Dr. Ahmed Tawfik
Eng. Rameez Barakat

Table of Contents

Project Overview	2
Business Rules and Assumptions.....	2
Our Assumptions:.....	2
System Design	3
Entity Relationship Diagram (ERD)	3
System Modules	4
Column Specifications Tables	4
Referential Integrity Constraint Table.....	11
GUI & System Screens.....	12
Login Page	12
Customer Dashboard	13
Loans Page	13
Transfer Interface	14
Transaction History Page	14
Staff Dashboard	15
Approval Requests Page.....	15
New Customer Registration.....	16
Team Management Page	16
SQL Queries and Functionalities	17
Queries Used	17
Conclusion.....	28
User Manual	28

Project Overview

The Banking Database Management System is a comprehensive database-driven application designed to simulate the core operations of a modern banking environment. The project focuses on building a secure, efficient, and scalable relational database capable of handling customer accounts, financial transactions, loans, cards, and employee management while maintaining data integrity and operational consistency.

The system was developed using relational database concepts and SQL to support both customer-side and staff-side banking functionalities. Customers are able to manage accounts, perform transfers, apply for loans and cards, monitor transactions, and update personal information. On the administrative side, employees can manage customers, approve financial requests, issue cards, monitor transactions, and oversee banking operations according to role-based permissions.

The database design emphasizes normalization, referential integrity, and security through the use of primary keys, foreign keys, constraints, validation rules, and transaction logging. Multiple interconnected tables were implemented to accurately represent real-world banking relationships such as joint accounts, customer-account ownership, employee supervision, and financial transaction tracking.

The project demonstrates the practical application of database management concepts in creating a structured and reliable banking system capable of supporting essential financial operations in a controlled environment.

Business Rules and Assumptions

Our Assumptions:

- Each customer can have multiple accounts, and a single account can be owned by multiple customers.
- One debit, and one credit card can be linked to one account. Customers can freeze/cancel their cards
- Internal (between a single customer's accounts) and external (between multiple customers' accounts) transfers are both allowed
- A customer can take out multiple loans. Loans must be taken out by a single customer (no joint loans)
- Employees register new customers and can freeze/delete customer accounts
- Employees can only edit accounts that were issued from the same branch
- Employees must approve loan requests and card requests

- Employees can have dependents
- Customers must be at least 16 years old to register.
- Customers must provide a unique 14-digit National ID.
- Customers can link a maximum of three phone numbers and phone numbers are strictly validated as mobile or landline formats.
- Addresses require distinct street, city, and governorate entries.
- Employee salaries must fall within preset job brackets.
- Staff view permissions depend strictly on corporate hierarchy.
- Staff are blocked from modifying their own roles or departments.
- Account 0 functions exclusively as the central bank vault.
- Accounts use status updates instead of hard row deletions to preserve history.

System Design

This section presents the overall structure of the banking database system, including the entities, relationships, and constraints used to organize and manage the data efficiently.

Entity Relationship Diagram (ERD)

The following ERD illustrates the relationships between the entities in the banking database system, including primary keys, foreign keys, and table connections.

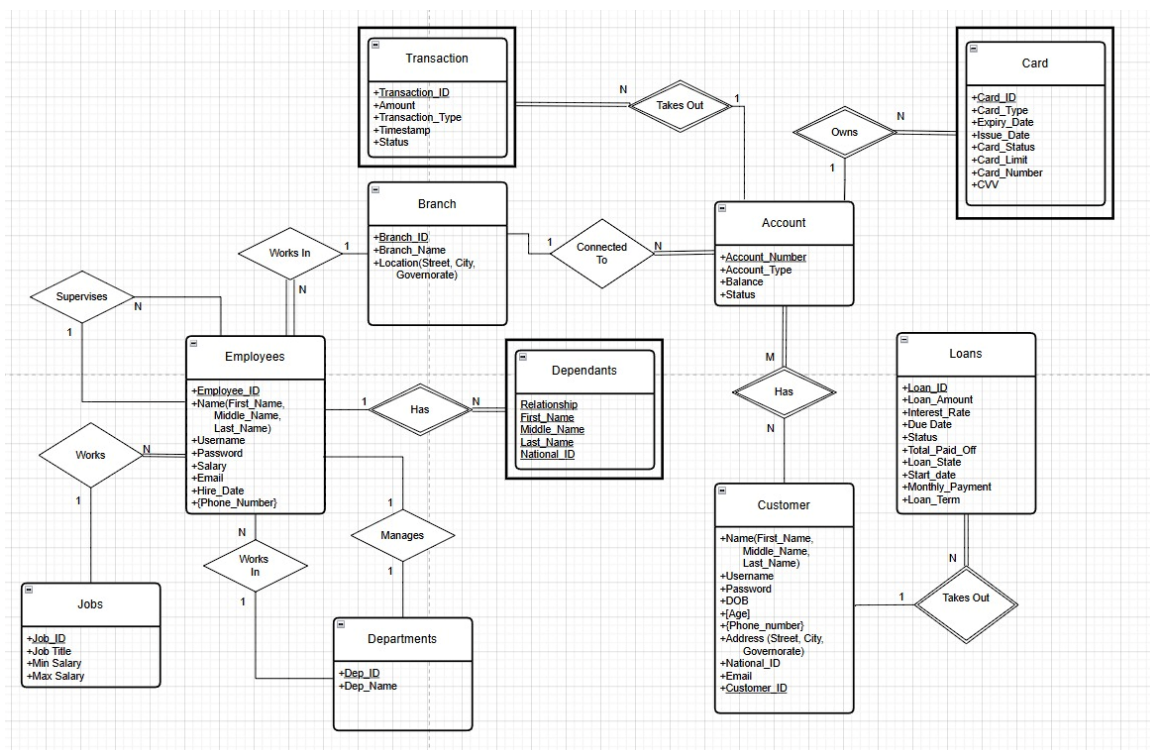


Figure 1: Banking Database ERD

System Modules

This section describes the structure of each table, including column names, data types, keys, constraints, and default values.

Column Specifications Tables

Customer

Column Name	Data Type	Key	Required	Default Value	Remarks
Customer_ID	NUMBER(10)	PK	Yes	None	Primary Key
First_name	VARCHAR2(50)	No	Yes	None	Customer first name
Middle_Name	VARCHAR2(50)	No	No	NULL	Customer middle name
Last_name	VARCHAR2(50)	No	Yes	None	Customer last name
DOB	DATE	No	Yes	None	Date of birth
Street	VARCHAR2(100)	No	Yes	None	Street address
City	VARCHAR2(50)	No	Yes	None	City
Governorate	VARCHAR2(50)	No	Yes	None	Governorate
National_ID	CHAR(14)	Unique	Yes	None	Unique national ID
Email	VARCHAR2(100)	Unique	Yes	None	Unique email
Username	VARCHAR2(32)	Unique	Yes	'TBD'	Login username
Password	VARCHAR2(60)	No	Yes	'TBD'	Login password

Table 1: Customer Table Column Specification

Branch

Column Name	Data Type	Key	Required	Default Value	Remarks
Branch_ID	NUMBER(10)	PK	Yes	None	Primary Key
Branch_Name	VARCHAR2(100)	No	Yes	None	Branch name
Street	VARCHAR2(100)	No	Yes	None	Street address
City	VARCHAR2(50)	No	Yes	None	City
Governorate	VARCHAR2(50)	No	Yes	None	Governorate

Table 2: Branch Table Column Specification

Department

Column Name	Data Type	Key	Required	Default Value	Remarks
Dep_ID	NUMBER(10)	PK	Yes	None	Primary Key
Dep_Name	VARCHAR2(100)	No	Yes	None	Department name

Table 3: Department Table Column Specification

Jobs

Column Name	Data Type	Key	Required	Default Value	Remarks
Job_ID	NUMBER(10)	PK	Yes	None	Primary Key
Job_Title	VARCHAR2(100)	No	Yes	None	Job title
Min_Salary	NUMBER(10,2)	No	Yes	None	Minimum salary
Max_Salary	NUMBER(10,2)	No	Yes	None	Maximum salary

Table 4: Jobs Table Column Specification

Employees

Column Name	Data Type	Key	Required	Default Value	Remarks
Employee_ID	NUMBER(10)	PK	Yes	None	Primary Key
First_Name	VARCHAR2(50)	No	Yes	None	Employee first name
Middle_Name	VARCHAR2(50)	No	No	NULL	Employee middle name
Last_Name	VARCHAR2(50)	No	Yes	None	Employee last name
Salary	NUMBER(10,2)	No	Yes	None	Must be > 0
Email	VARCHAR2(100)	Unique	Yes	None	Unique email
Hire_Date	DATE	No	Yes	None	Hiring date
Username	VARCHAR2(32)	Unique	Yes	'TBD'	Login username
Password	VARCHAR2(60)	No	Yes	'TBD'	Login password
Branch_ID	NUMBER(10)	FK	Yes	None	References Branch
Job_ID	NUMBER(10)	FK	Yes	None	References Jobs
Dep_ID	NUMBER(10)	FK	Yes	None	References Department
Supervisor_ID	NUMBER(10)	FK	No	NULL	References Employees (ON DELETE SET NULL)

Table 5: Employees Table Column Specification

Account

Column Name	Data Type	Key	Required	Default Value	Remarks
Account_Number	NUMBER(12)	PK	Yes	None	Primary Key

Column Name	Data Type	Key	Required	Default Value	Remarks
Account_Type	VARCHAR2(20)	No	Yes	None	Must be 'Savings', 'Retail', 'Corporate', 'Student', or 'Joint'
Balance	NUMBER(15,2)	No	Yes	None	Must be ≥ 0
Status	VARCHAR2(20)	No	Yes	None	Must be 'Active', 'Inactive', or 'Closed'
Branch_ID	NUMBER(10)	FK	Yes	None	References Branch

Table 6: Account Table Column Specification

Loan

Column Name	Data Type	Key	Required	Default Value	Remarks
Loan_ID	NUMBER(10)	PK	Yes	None	Primary Key
Customer_ID	NUMBER(10)	FK	Yes	None	References Customer
Loan_Amount	NUMBER(15,2)	No	Yes	None	Must be > 0
Interest_Rate	NUMBER(5,2)	No	Yes	None	Must be ≥ 0
Due_Date	DATE	No	Yes	None	Loan due date
Status	VARCHAR2(20)	No	Yes	None	Must be 'Pending', 'Approved', 'Rejected', 'Paid', or 'Overdue'
Loan_Term	NUMBER(3,0)	No	No	NULL	Loan term duration
Monthly_Payment	NUMBER(15,2)	No	No	NULL	Monthly payment amount
Total_Paid_Off	NUMBER(15,2)	No	Yes	0	Amount of loan paid off
Loan_State	VARCHAR2(20)	No	Yes	'ACTIVE'	Must be 'ACTIVE' or 'PAID'

Column Name	Data Type	Key	Required	Default Value	Remarks
-------------	-----------	-----	----------	---------------	---------

Start_Date	DATE	No	Yes	SYSDATE	When loan was approved
------------	------	----	-----	---------	------------------------

Table 7: Loan Table Column Specification

Card

Column Name	Data Type	Key	Required	Default Value	Remarks
-------------	-----------	-----	----------	---------------	---------

Card_ID	NUMBER(10)	PK	Yes	None	Primary Key
---------	------------	-----------	-----	------	-------------

Account_Number	NUMBER(12)	FK	Yes	None	References Account
----------------	------------	-----------	-----	------	--------------------

Card_Type	VARCHAR2(20)	No	Yes	None	Must be 'Debit', 'Credit', or 'Prepaid'
-----------	--------------	----	-----	------	---

Expiry_Date	DATE	No	Yes	None	Expiry date
-------------	------	----	-----	------	-------------

Issue_Date	DATE	No	Yes	None	Issue date
------------	------	----	-----	------	------------

Card_Status	VARCHAR2(20)	No	Yes	None	Must be 'Active', 'Expired', 'Blocked', 'Suspended', or 'Pending'
-------------	--------------	----	-----	------	---

Card_Limit	NUMBER(15,2)	No	No	NULL	Required for 'Credit' cards; must be NULL for others
------------	--------------	----	----	------	--

Card_Number	CHAR(16)	Unique	Yes	None	Unique card number
-------------	----------	--------	-----	------	--------------------

CVV	CHAR(3)	No	Yes	None	Security code
-----	---------	----	-----	------	---------------

Table 8: Card Table Column Specification

Bank_Transaction

Column Name	Data Type	Key	Required	Default Value	Remarks
-------------	-----------	-----	----------	---------------	---------

Transaction_ID	NUMBER(12)	PK	Yes	None	Primary Key
----------------	------------	-----------	-----	------	-------------

Column Name	Data Type	Key	Required	Default Value	Remarks
Sender_Account_Number	NUMBER(12)	FK	No	NULL	References Account (NULL if 'Deposit')
Receiver_Account_Number	NUMBER(12)	FK	No	NULL	References Account (NULL if 'Withdrawal')
Amount	NUMBER(15,2)	No	Yes	None	Must be > 0
Transaction_Type	VARCHAR2(10)	No	Yes	None	Must be 'Deposit', 'Withdrawal', 'Transfer', or 'Payment'
Transaction_Time	TIMESTAMP	No	Yes	None	Timestamp of transaction
Status	VARCHAR2(10)	No	Yes	None	Must be 'Pending', 'Completed', 'Failed', or 'Canceled'

Table 9: Bank_Transaction Table Column Specification

Dependants

Column Name	Data Type	Key	Required	Default Value	Remarks
Employee_ID	NUMBER(10)	PK, FK	Yes	None	References Employees (ON DELETE CASCADE)
National_ID	VARCHAR2(14)	PK, Unique	Yes	None	Unique National ID
First_Name	VARCHAR2(50)	No	Yes	None	First name
Relationship	VARCHAR2(50)	No	Yes	None	Relationship type
Middle_Name	VARCHAR2(50)	No	No	NULL	Middle name
Last_Name	VARCHAR2(50)	No	Yes	None	Last name

Table 10: Dependants Table Column Specification

Customer_Phone

Column Name	Data Type	Key	Required	Default Value	Remarks
Customer_ID	NUMBER(10)	PK, FK	Yes	None	References Customer (ON DELETE CASCADE)
Customer_Phone	NUMBER(11)	PK	Yes	None	Phone number

Table 11: Customer_Phone Table Column Specification

Employee_Phone

Column Name	Data Type	Key	Required	Default Value	Remarks
Employee_ID	NUMBER(10)	PK, FK	Yes	None	References Employees (ON DELETE CASCADE)
Employee_Phone	NUMBER(11)	PK	Yes	None	Phone number

Table 12: Employee_Phone Table Column Specification

Customer_Account

Column Name	Data Type	Key	Required	Default Value	Remarks
Customer_ID	NUMBER(10)	PK, FK	Yes	None	References Customer (ON DELETE CASCADE)
Account_Number	NUMBER(12)	PK, FK	Yes	None	References Account (ON DELETE CASCADE)

Table 13: Customer_Account Table Column Specification

Department_Managers

Column Name	Data Type	Key	Required	Default Value	Remarks
Employee_ID	NUMBER(10)	PK, FK	Yes	None	References Employees (ON DELETE CASCADE)

Column Name	Data Type	Key	Required	Default Value	Remarks
Dep_ID	NUMBER(10)	PK, FK	Yes	None	References Department (ON DELETE CASCADE)

Table 14: Department_Managers Table Column Specification

Referential Integrity Constraint Table

Referential integrity constraints were implemented to maintain consistency between related tables and prevent invalid references within the database.

Relationship		Referential Integrity Constraint	Cascading Behavior	
Referenced Table	Referencing Table		On Update	On Delete
Branch	Employees	Branch_ID	No Action	No Action
Jobs	Employees	Job_ID	No Action	No Action
Department	Employees	Dep_ID	No Action	No Action
Employees	Employees	Supervisor_ID	No Action	SET NULL
Branch	Account	Branch_ID	No Action	No Action
Customer	Loan	Customer_ID	No Action	No Action
Account	Card	Account_Number	No Action	No Action
Account	Bank_Transaction	Sender_Account_Number	No Action	No Action
Account	Bank_Transaction	Receiver_Account_Number	No Action	No Action
Employees	Dependants	Employee_ID	No Action	CASCADE
Customer	Customer_Phone	Customer_ID	No Action	CASCADE

Relationship		Referential Integrity Constraint	Cascading Behavior	
Referenced Table	Referencing Table		On Update	On Delete
Employees	Employee_Phone	Employee_ID	No Action	CASCADE
Customer	Customer_Account	Customer_ID	No Action	CASCADE
Account	Customer_Account	Account_Number	No Action	CASCADE
Employees	Department_Managers	Employee_ID	No Action	CASCADE
Department	Department_Managers	Dep_ID	No Action	CASCADE

Table 15: Referential Integrity Constraint Table

GUI & System Screens

A graphical user interface (GUI) was developed to allow customers and employees to interact with the banking system. The following screenshots demonstrate the main system interfaces and functionalities

Login Page

The login interface allows customers and employees to securely access the banking system using their usernames and passwords.

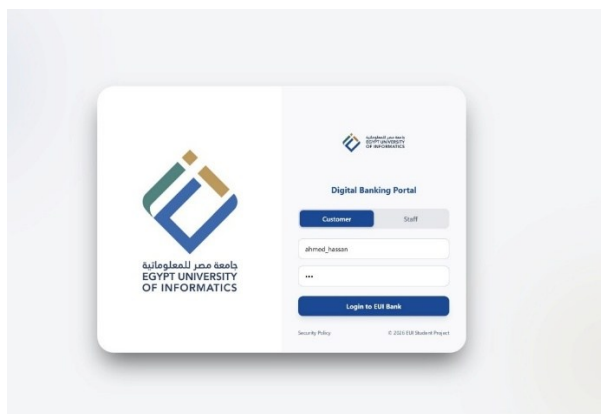


Figure 2: Customer Login Page

Customer Dashboard

The customer dashboard displays account information, balances, recent transactions, and quick access to banking services.

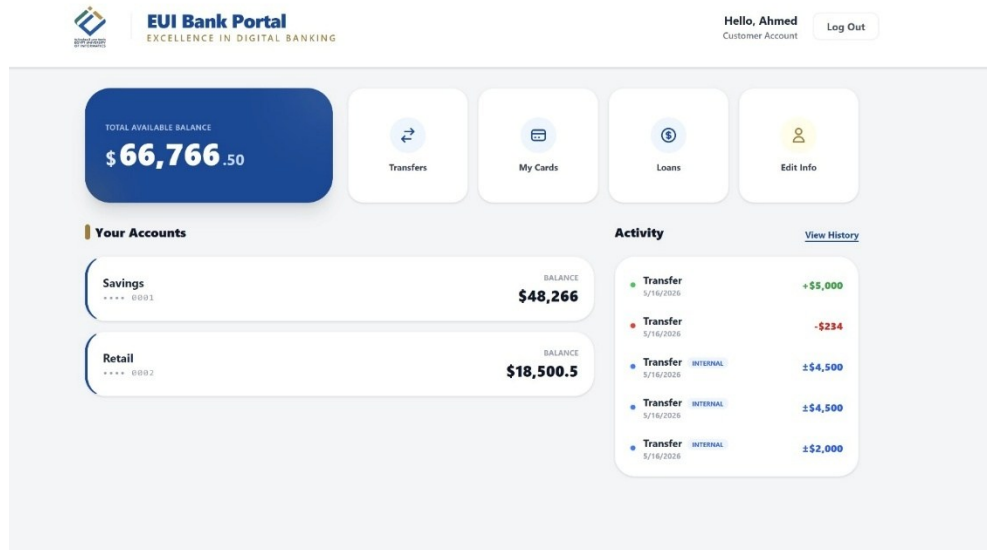


Figure 3: Customer Dashboard

Loans Page

The loans page displays the customer's active and pending loans, including loan amounts, payment status, and due dates.

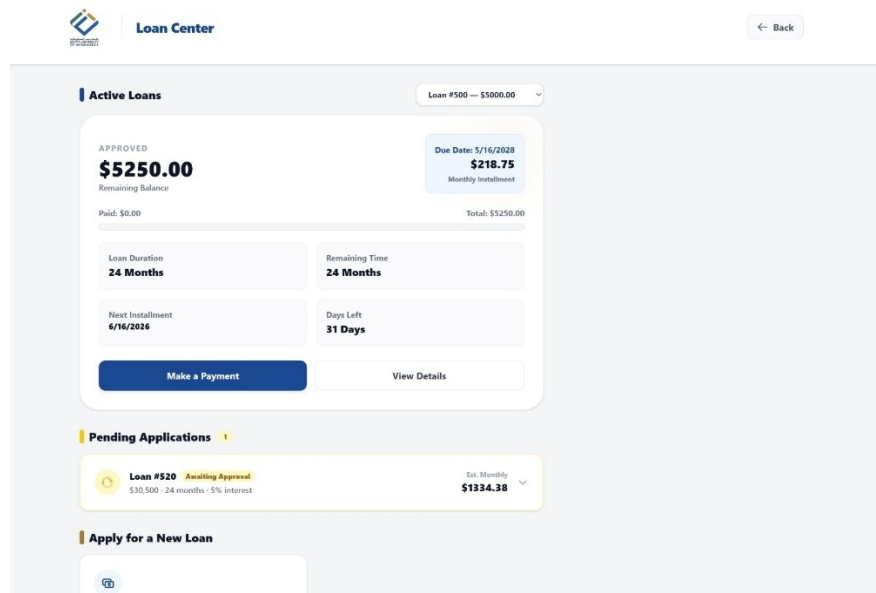
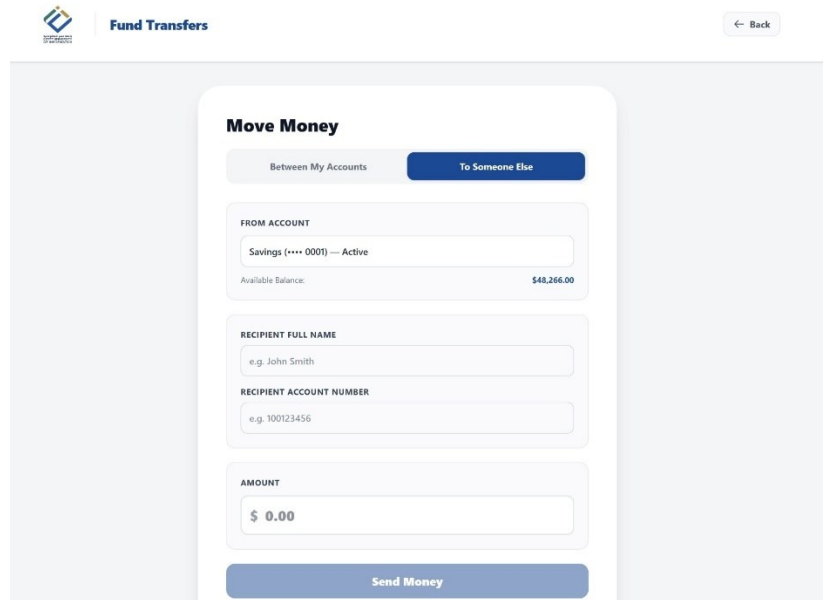


Figure 4: Loans Page

Transfer Interface

The transfer interface allows customers to securely transfer funds between accounts after validating balances and account statuses.

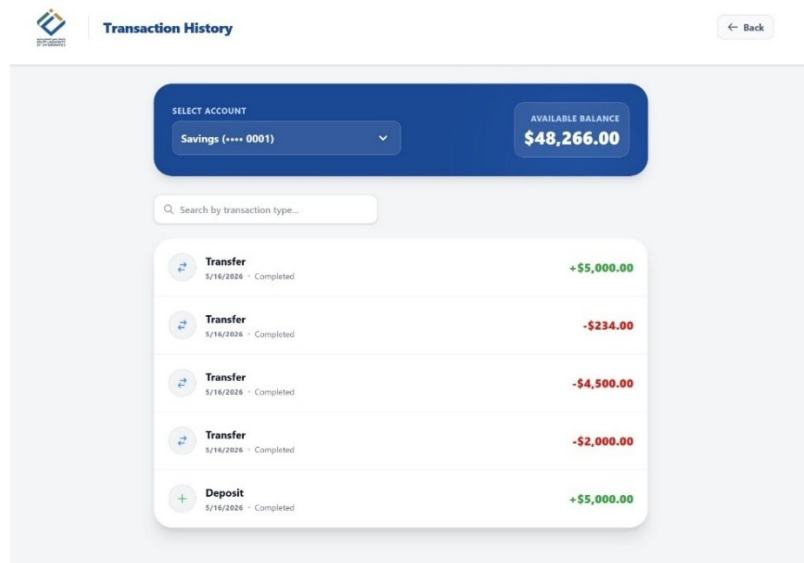


The screenshot shows the 'Fund Transfers' page with a 'Move Money' form. The form has two tabs: 'Between My Accounts' (selected) and 'To Someone Else'. The 'FROM ACCOUNT' section shows 'Savings (*** 0001) — Active' with an available balance of \$48,266.00. The 'RECIPIENT FULL NAME' field contains 'e.g. John Smith'. The 'RECIPIENT ACCOUNT NUMBER' field contains 'e.g. 100123456'. The 'AMOUNT' field shows '\$ 0.00'. A 'Send Money' button is at the bottom.

Figure 5: Money Transfer Interface

Transaction History Page

The transaction history page displays recent transfers and banking activities associated with the customer's accounts.



The screenshot shows the 'Transaction History' page. At the top, there's a 'SELECT ACCOUNT' dropdown menu showing 'Savings (**** 0001)' and an 'AVAILABLE BALANCE' of \$48,266.00. Below this is a search bar labeled 'Search by transaction type...'. The main content is a list of transactions:

Transaction Type	Date	Status	Amount
Transfer	5/16/2026	Completed	+\$5,000.00
Transfer	5/16/2026	Completed	-\$234.00
Transfer	5/16/2026	Completed	-\$4,500.00
Transfer	5/16/2026	Completed	-\$2,000.00
Deposit	5/16/2026	Completed	+\$5,000.00

Figure 6: Transaction History Page

Staff Dashboard

The staff dashboard provides employees with quick access to customer management, approvals, transactions, and administrative operations.

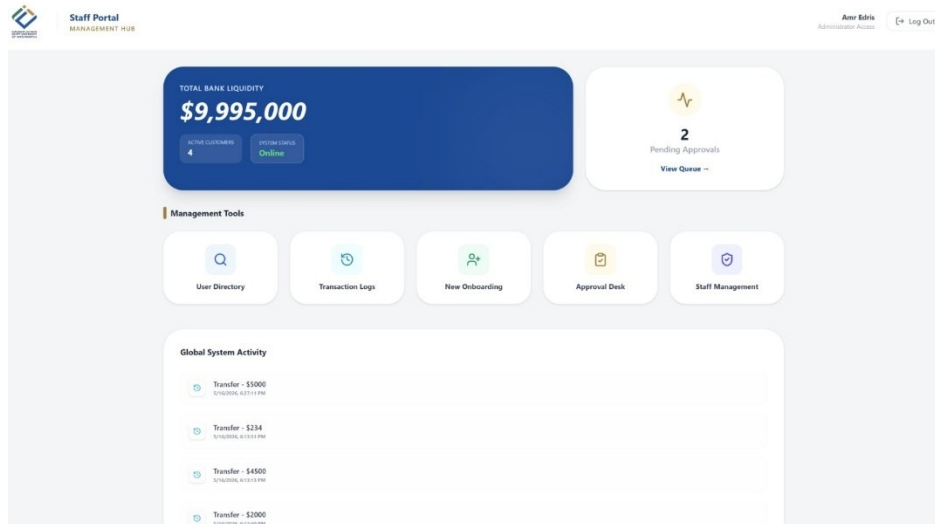


Figure 7: Staff Dashboard

Approval Requests Page

The approvals page allows employees to review and process pending loan and card applications.

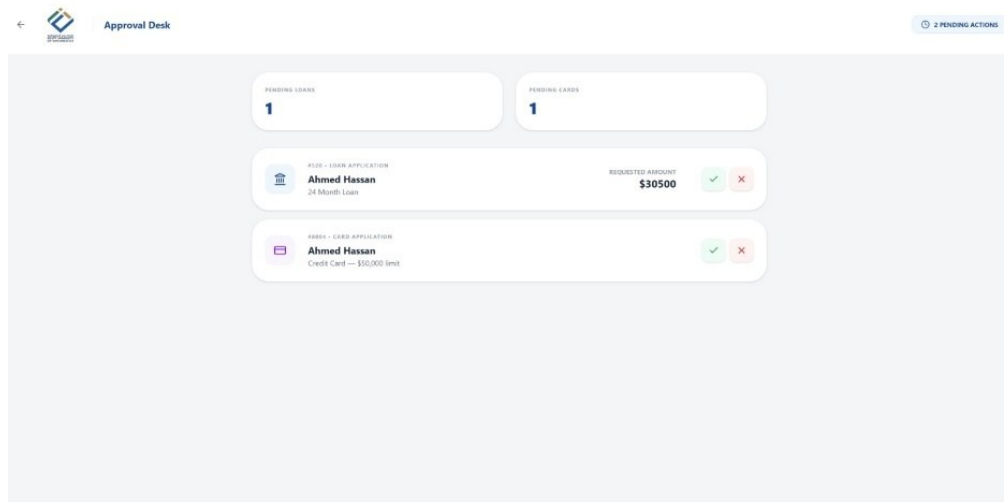
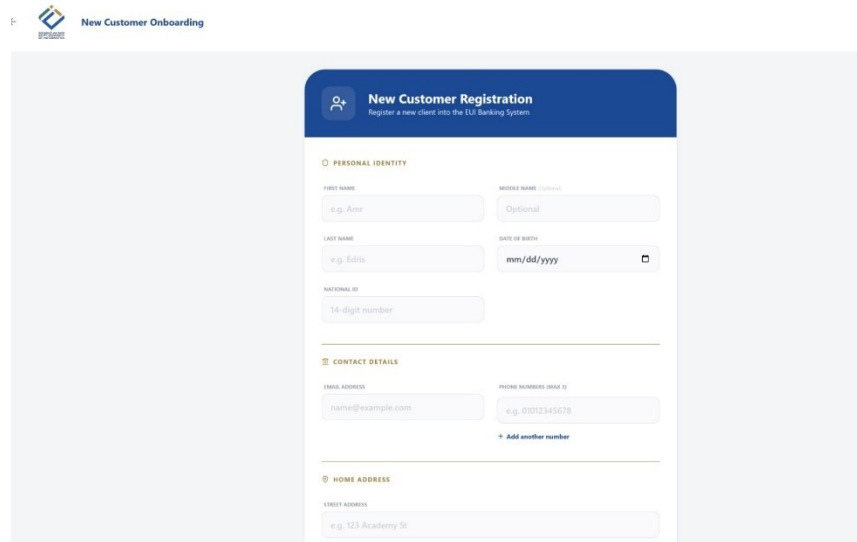


Figure 8: Approval Requests Page

New Customer Registration

The customer registration interface allows employees to create new customer accounts by entering personal and banking information.



The screenshot shows a web interface for "New Customer Onboarding". The main form is titled "New Customer Registration" with the subtitle "Register a new client into the UI Banking System". The form is divided into three sections: "PERSONAL IDENTITY", "CONTACT DETAILS", and "HOME ADDRESS".

PERSONAL IDENTITY

- FIRST NAME: e.g. Amr
- MIDDLE NAME (Optional): Optional
- LAST NAME: e.g. Edris
- DATE OF BIRTH: mm/dd/yyyy
- NATIONAL ID: 14-digit number

CONTACT DETAILS

- EMAIL ADDRESS: name@example.com
- PHONE NUMBERS (MAX 10): e.g. 01012345678, + Add another number

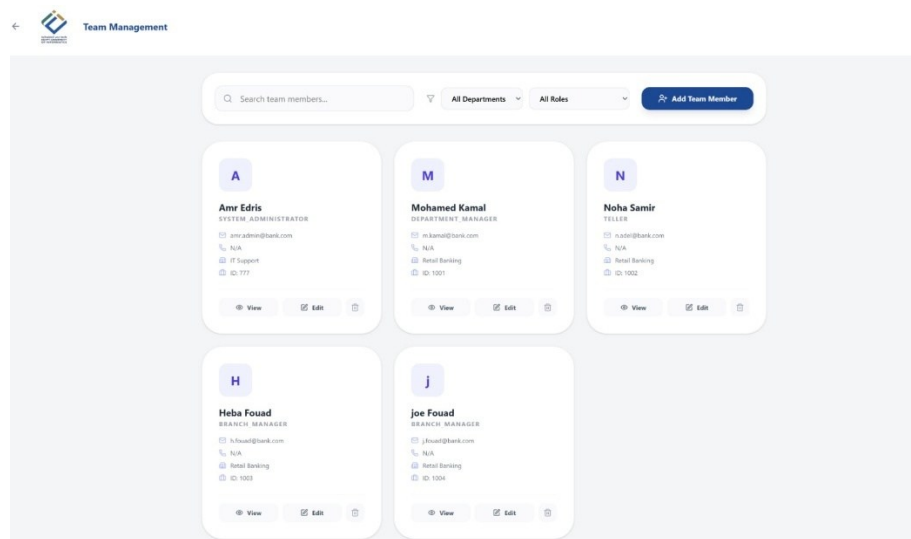
HOME ADDRESS

- STREET ADDRESS: e.g. 123 Academy St

Figure 9: Customer Registration Screen

Team Management Page

The staff management page enables administrators to manage employee information, roles, and permissions within the system.



The screenshot shows a web interface for "Team Management". At the top, there is a search bar "Search team members...", a dropdown for "All Departments", a dropdown for "All Roles", and a button "Add Team Member". Below this, there are five employee cards arranged in a grid. Each card displays the employee's name, title, email, phone number, and ID, along with "View", "Edit", and "Delete" icons.

Name	Title	Email	Phone	ID
Amr Edris	SYSTEM ADMINISTRATOR	amradmin@bank.com	N/A	IT Support
Mohamed Kamal	DEPARTMENT MANAGER	mkamal@bank.com	N/A	Retail Banking
Noha Samir	TELLER	n.sadr@bank.com	N/A	Retail Banking
Heba Fouad	BRANCH MANAGER	h.fouad@bank.com	N/A	Retail Banking
Joe Fouad	BRANCH MANAGER	j.fouad@bank.com	N/A	Retail Banking

Figure 10: Team Management Page

SQL Queries and Functionalities

SQL queries were implemented to support the main functionalities of the banking system, including authentication, transfers, loans, card management, and customer operations.

Queries Used

Transfer

```
`SELECT balance, status FROM account WHERE account_number = :acc`
```

Fetches the sender's account balance and status before a transfer to check they have enough funds and that their account is active.

```
`SELECT status FROM account WHERE account_number = :acc`
```

Checks that the recipient's account exists and is active before accepting a transfer into it.

```
`UPDATE account SET balance = balance - :amount WHERE account_number = :fromAcc`
```

Deducts the transferred amount from the sender's account balance.

```
`UPDATE account SET balance = balance + :amount WHERE account_number = :toAcc`
```

Adds the transferred amount to the recipient's account balance.

```
'INSERT INTO bank_transaction (transaction_id, transaction_type, amount, transaction_time, sender_account_number, receiver_account_number, status) VALUES (transaction_seq.NEXTVAL, 'Transfer', :amount, SYSDATE, :fromAcc, :toAcc, 'Completed')'
```

Logs the completed transfer as a permanent record in the transactions table.

Customer Login

```
'SELECT c.customer_id, c.first_name, c.last_name, NVL(SUM(a.balance), 0) AS total_balance FROM customer c LEFT JOIN customer_account ca ON c.customer_id = ca.customer_id LEFT JOIN account a ON ca.account_number = a.account_number WHERE c.username = :us AND c.password = :pass GROUP BY c.customer_id, c.first_name, c.last_name'
```

Verifies the customer's credentials and returns their name and total balance across all accounts to display on the dashboard.

Staff Login

```
'SELECT employee_id, first_name, last_name, email, branch_id, job_id, dep_id, username FROM employees WHERE username = :us AND password = :pass'
```

Verifies the staff member's credentials and returns their profile information, including their role and department, to control what they can see in the staff dashboard.

Accounts

```
'SELECT a.account_number, a.account_type, a.balance, a.status FROM customer_account ca JOIN account a ON ca.account_number = a.account_number WHERE ca.customer_id = :id'
```

Retrieves all accounts belonging to the logged-in customer to display on their accounts page.

```
'SELECT account_number, account_type, balance, status, branch_id FROM account WHERE account_number = :acc'
```

Fetches the full details of a specific account when a customer selects it to view.

Transactions

```
'SELECT * FROM (SELECT bt.transaction_id, bt.transaction_type, bt.amount, bt.transaction_time, bt.sender_account_number, bt.receiver_account_number, bt.status FROM bank_transaction bt JOIN customer_account ca ON bt.sender_account_number = ca.account_number OR bt.receiver_account_number = ca.account_number WHERE ca.customer_id = :id ORDER BY bt.transaction_time DESC) WHERE ROWNUM <= 5'
```

Gets the 5 most recent transactions linked to the logged-in customer across all their accounts, shown on the dashboard.

```
'SELECT transaction_id, transaction_type, amount, transaction_time, sender_account_number, receiver_account_number, status FROM bank_transaction WHERE sender_account_number = :acc OR receiver_account_number = :acc ORDER BY transaction_time DESC'
```

Gets the full transaction history for a specific account, whether the account was the sender or receiver.

Cards

```
'SELECT c.CARD_ID, c.CARD_TYPE, c.CARD_NUMBER, c.EXPIRY_DATE, c.CVV, c.CARD_STATUS, c.ACCOUNT_NUMBER, c.CARD_LIMIT, cust.FIRST_NAME || ' ' || cust.LAST_NAME AS CARDHOLDER FROM CARD c JOIN CUSTOMER_ACCOUNT ca
```

```
ON c.ACCOUNT_NUMBER = ca.ACCOUNT_NUMBER JOIN CUSTOMER cust ON  
ca.CUSTOMER_ID = cust.CUSTOMER_ID WHERE ca.CUSTOMER_ID = :id'
```

Gets all cards owned by the logged-in customer by linking each card through the account to the customer.

```
'SELECT c.CARD_ID, c.CARD_TYPE, c.CARD_NUMBER, c.EXPIRY_DATE, c.CVV,  
c.CARD_STATUS, c.ACCOUNT_NUMBER, c.CARD_LIMIT, cust.FIRST_NAME || ' '  
|| cust.LAST_NAME AS CARDHOLDER FROM CARD c JOIN CUSTOMER_ACCOUNT ca  
ON c.ACCOUNT_NUMBER = ca.ACCOUNT_NUMBER JOIN CUSTOMER cust ON  
ca.CUSTOMER_ID = cust.CUSTOMER_ID WHERE c.ACCOUNT_NUMBER = :acc'
```

Gets all cards linked to one specific account number, used when viewing a single account's cards.

```
'SELECT CARD_STATUS FROM CARD WHERE CARD_ID = :id'
```

Checks the current status of a card before allowing a status change, specifically to block changes to permanently suspended cards.

```
'UPDATE CARD SET CARD_STATUS = :status WHERE CARD_ID = :id'
```

Updates a card's status (e.g. Active, Blocked, Suspended) after the customer requests a change.

Card Application

```
'SELECT account_number FROM account WHERE account_number = :acc'
```

Confirms the account number the customer entered actually exists before applying for a card on it.

```
'SELECT card_id FROM card WHERE account_number = :acc AND card_type =  
:type AND card_status IN ('Active', 'Pending')'
```

Prevents a customer from applying for a card type they already have active or pending approval on the same account.

```
'SELECT NVL(MAX(card_id), 5000) + 1 AS next_id FROM card'
```

Calculates the next available card ID. If the card table is empty it starts at 5001, otherwise it takes the current highest ID and adds 1.

```
'INSERT INTO card (card_id, card_type, card_limit, card_number,  
issue_date, expiry_date, cvv, card_status, account_number) VALUES  
(:id, :type, :limit, :num, SYSDATE, ADD_MONTHS(SYSDATE, 48), :cvv,  
'Pending', :accNum)'
```

Creates a new card application record. The expiry date is automatically set 4 years (48 months) from today and the status starts as Pending until staff approve it.

Loans

```
'SELECT * FROM Loan WHERE customer_id = :id AND loan_state = 'ACTIVE'  
AND status = 'Approved' ORDER BY start_date DESC'
```

Retrieves all currently active and approved loans for the logged-in customer, newest first.

Loan Application Validation

```
'SELECT dob FROM customer WHERE customer_id = :cid'
```

Fetches the customer's date of birth to verify they are at least 18 years old before allowing a loan application.

```
'SELECT COUNT(*) AS active_count FROM customer_account ca JOIN account  
a ON ca.account_number = a.account_number WHERE ca.customer_id = :cid  
AND a.status = 'Active''
```

Checks that the customer has at least one active bank account, which is required to receive the loan funds if approved.

```
'SELECT COUNT(*) AS active_loans FROM Loan WHERE customer_id = :cid  
AND (status = 'Pending' OR loan_state = 'ACTIVE')'
```

Ensures the customer does not already have a pending or active loan, since only one loan is allowed at a time.

```
'INSERT INTO Loan (loan_id, customer_id, loan_amount, interest_rate,  
due_date, status, loan_term, total_paid_off, loan_state, start_date,  
monthly_payment) VALUES (loan_seq.NEXTVAL, :cid, :amt, :rate,  
ADD_MONTHS(SYSDATE, :term), 'Pending', :term, 0, 'ACTIVE', SYSDATE,  
:monthly)'
```

Submits a new loan application. The due date is calculated by adding the loan term in months to today, and the total paid off starts at 0.

```
SELECT COUNT(*) AS active_loans FROM Loan WHERE customer_id = :cid AND  
(status = 'Pending' OR loan_state = 'ACTIVE')
```

Ensures the customer does not already have a pending or active loan before submitting a new application, since only one loan is allowed at a time. The /loans/apply route performs the age check and active-account check but skips this duplicate-loan guard entirely.

Loan Payment

```
'SELECT balance FROM account WHERE account_number = :acc'
```

Gets the customer's current account balance to confirm they have enough to cover the payment amount.

```
'SELECT loan_amount, interest_rate, total_paid_off FROM Loan WHERE  
loan_id = :lid'
```

Retrieves the loan's original amount, interest rate, and how much has already been paid to calculate the remaining balance and prevent overpayment.

```
'UPDATE account SET balance = balance - :amount WHERE account_number =  
:acc'
```

Deducts the payment amount from the customer's bank account.

```
'UPDATE Loan SET total_paid_off = :newTotalPaid WHERE loan_id = :lid'
```

Updates the loan record with the new cumulative total the customer has paid back.

```
'UPDATE Loan SET loan_state = 'PAID' WHERE loan_id = :lid'
```

Runs only if the customer's payments now cover the full amount owed including interest. Changes the loan state from ACTIVE to PAID.

Customer Profile

```
'SELECT c.first_name, c.last_name, c.email, c.street, c.city,  
c.governorate, cp.customer_phone AS phone FROM customer c LEFT JOIN  
customer_phone cp ON c.customer_id = cp.customer_id WHERE  
c.customer_id = :id'
```

Gets all personal details and phone number for the logged-in customer. The LEFT JOIN ensures their info still shows even if they have no phone number registered.

```
'UPDATE Customer SET First_Name = :firstName, Last_Name = :lastName,  
Email = :email, Street = :street, City = :city, Governorate =  
:governorate WHERE Customer_ID = :customerId'
```

Saves the customer's updated personal information.

```
'SELECT CUSTOMER_ID FROM CUSTOMER_PHONE WHERE CUSTOMER_PHONE = :phone  
AND CUSTOMER_ID != :customerId'
```

Checks whether the new phone number the customer entered is already registered to a different customer, to prevent duplicate phone numbers.

```
'DELETE FROM CUSTOMER_PHONE WHERE CUSTOMER_ID = :customerId'
```

Removes the customer's existing phone number before inserting the updated one.

```
'INSERT INTO CUSTOMER_PHONE (CUSTOMER_ID, CUSTOMER_PHONE) VALUES  
(:customerId, :phone)'
```

Saves the customer's new phone number.

```
'SELECT password FROM customer WHERE customer_id = :id'
```

Retrieves the current password from the database to verify the customer typed their old password correctly before allowing a change.

```
'UPDATE customer SET password = :newPassword WHERE customer_id = :id'
```

Replaces the old password with the new one.

Admin Dashboard

```
'SELECT NVL(SUM(balance), 0) AS TOTAL FROM account'
```

Adds up the balances of all accounts in the bank to show the total liquidity figure on the admin dashboard.

```
'SELECT COUNT(*) AS TOTAL FROM customer'
```

Counts the total number of registered customers.

```
'SELECT (SELECT COUNT(*) FROM Loan WHERE status = 'Pending') + (SELECT COUNT(*) FROM card WHERE card_status = 'Pending') AS TOTAL FROM dual'
```

Counts the total number of items waiting for staff approval by combining pending loans and pending card applications into one number.

```
'SELECT * FROM (SELECT transaction_id, transaction_type, amount, transaction_time FROM bank_transaction ORDER BY transaction_time DESC) WHERE ROWNUM <= 5'
```

Fetches the 5 most recent transactions across the entire bank for the admin activity feed.

```
SELECT NVL(SUM(balance), 0) AS TOTAL FROM account
```

Adds up the balances of **all** accounts in the bank to show total liquidity. The /admin/stats route instead runs SELECT balance AS TOTAL FROM account WHERE account_number = 0, which only reads the bank's reserve vault (account 0) rather than the aggregate of every account.

Approvals

```
'SELECT l.loan_id AS ID, l.loan_amount, l.loan_term, c.first_name || ' || c.last_name AS customer_name, 'loan' AS request_type FROM loan l JOIN customer c ON l.customer_id = c.customer_id WHERE l.status = 'Pending''
```

Gets all loan applications currently waiting for staff approval, along with the applicant's name.

```
'SELECT cd.card_id AS ID, cd.card_type, cd.card_limit, c.first_name || ' || c.last_name AS customer_name, 'card' AS request_type FROM card cd JOIN customer_account ca ON cd.account_number = ca.account_number
```

```
JOIN customer c ON ca.customer_id = c.customer_id WHERE cd.card_status = 'Pending''
```

Gets all card applications currently waiting for staff approval, along with the cardholder's name.

Processing a Loan

```
'SELECT customer_id, loan_amount FROM loan WHERE loan_id = :loanId'
```

Fetches the loan's amount and the customer it belongs to before processing an approval or rejection.

```
'UPDATE loan SET status = 'Rejected' WHERE loan_id = :loanId'
```

Marks a loan application as rejected.

```
'SELECT balance FROM account WHERE account_number = 0'
```

Checks the bank's own reserve account (account 0) to ensure it has enough funds to cover the loan before approving it.

```
'SELECT a.account_number FROM customer_account ca JOIN account a ON ca.account_number = a.account_number WHERE ca.customer_id = :customerId AND a.status = 'Active' AND ROWNUM = 1'
```

Finds one active account belonging to the customer to deposit the approved loan funds into.

```
'UPDATE loan SET status = 'Approved' WHERE loan_id = :loanId'
```

Marks the loan as officially approved.

```
'UPDATE account SET balance = balance - :amount WHERE account_number = 0'
```

Deducts the loan amount from the bank's reserve account (the vault).

```
'UPDATE account SET balance = balance + :amount WHERE account_number = :accNum'
```

Deposits the loan funds into the customer's account.

```
'INSERT INTO bank_transaction (transaction_id, transaction_type, amount, transaction_time, sender_account_number, receiver_account_number, status) VALUES (transaction_seq.NEXTVAL, 'Deposit', :amount, SYSDATE, 0, :accNum, 'Completed)'
```

Logs the loan disbursement as a bank deposit transaction from account 0 to the customer.

Processing a Card

```
'UPDATE card SET card_status = :status WHERE card_id = :cardId'
```

Sets the card status to Active (approved) or Suspended (rejected) based on the staff member's decision.

Staff — Managing Staff Members

```
'SELECT job_id FROM employees WHERE employee_id = :empId'
```

Looks up the requester's job role to determine what level of access they have when viewing the staff list.

```
'SELECT COUNT(*) AS sup_count FROM employees WHERE supervisor_id = :empId'
```

Checks whether a manager actually supervises any employees before allowing them to view the staff list.

```
'SELECT e.employee_id, e.first_name, e.last_name, e.email,  
e.supervisor_id, e.salary, e.dep_id, e.username, j.job_title FROM  
employees e JOIN jobs j ON e.job_id = j.job_id ORDER BY e.employee_id'
```

Retrieves the full staff list, joined with the jobs table to include each employee's job title. Results are filtered based on the requester's role.

```
'SELECT NVL(MAX(employee_id), 1000) + 1 AS next_id FROM employees'
```

Generates the next available employee ID when creating a new staff member.

```
'SELECT job_id FROM jobs WHERE job_title = :role'
```

Translates a role name (e.g. "Teller") into the corresponding job ID stored in the database.

```
'SELECT dep_id FROM employees WHERE employee_id = :empId'
```

If no department is specified when creating a new staff member, this inherits the department from their supervisor.

```
'INSERT INTO employees (employee_id, first_name, middle_name,  
last_name, email, salary, hire_date, username, password, branch_id,  
job_id, dep_id, supervisor_id) VALUES (:employeeId, :firstName,  
:middleName, :lastName, :email, :salary, SYSDATE, :username,  
:password, :branchId, :jobId, :depId, :supervisorId)'
```

Creates a new employee record with today as their hire date.

```
'INSERT INTO employee_phone (employee_id, employee_phone) VALUES  
(:empId, :phone)'
```

Adds a phone number for the new employee. This runs once per phone number if multiple are provided.

```
'UPDATE employees e SET e.first_name = :firstName, e.last_name = :lastName, e.email = :email, e.job_id = (SELECT job_id FROM jobs WHERE job_title = :role), e.salary = :salary, e.password = :password WHERE e.employee_id = :employeeId'
```

Updates a staff member's details. The role name is converted to a job ID inline using a subquery.

```
'DELETE FROM employee_phone WHERE employee_id = :empId'
```

Clears all existing phone numbers for a staff member before inserting the updated ones.

```
'DELETE FROM employees WHERE employee_id = :employeeId'
```

Permanently removes a staff member from the system.

```
SELECT COUNT(*) AS sup_count FROM employees WHERE supervisor_id = :empId
```

Checks whether a manager actually supervises any employees before allowing them to view the staff list. The /staff route resolves the requester's job ID and branches on it, but never runs this supervisee-count query that the documentation specifies as part of the access-control flow.

Staff — Managing Customers

```
'SELECT c.customer_id, c.first_name, c.last_name, c.national_id, NVL(SUM(a.balance), 0) AS balance, UPPER(NVL(MAX(a.status), 'Inactive')) AS status FROM customer c LEFT JOIN customer_account ca ON c.customer_id = ca.customer_id LEFT JOIN account a ON ca.account_number = a.account_number GROUP BY c.customer_id, c.first_name, c.last_name, c.national_id ORDER BY c.customer_id'
```

Gets a summary of every customer with their total balance and overall account status, used in the staff customer management table.

```
'SELECT c.customer_id, c.first_name, c.last_name, c.email, c.street, c.city, c.governorate, c.dob, cp.customer_phone FROM customer c LEFT JOIN customer_phone cp ON c.customer_id = cp.customer_id WHERE c.customer_id = :id'
```

Gets full personal details of a specific customer for the staff profile view, including their date of birth.

```
'SELECT a.account_number, a.account_type, a.balance, a.status FROM
customer_account ca JOIN account a ON ca.account_number =
a.account_number WHERE ca.customer_id = :id'
```

Gets all accounts belonging to a specific customer, shown on their staff-side profile.

```
'SELECT * FROM (SELECT transaction_id, transaction_type, amount,
transaction_time, status FROM bank_transaction bt WHERE
bt.sender_account_number IN (SELECT account_number FROM
customer_account WHERE customer_id = :id) OR
bt.receiver_account_number IN (SELECT account_number FROM
customer_account WHERE customer_id = :id) ORDER BY transaction_time
DESC) WHERE ROWNUM <= 10'
```

Gets the 10 most recent transactions for a specific customer across all their accounts, shown on their staff-side profile.

```
'SELECT NVL(MAX(customer_id), 1000) + 1 AS next_id FROM customer'
```

Generates the next available customer ID when onboarding a new customer.

```
'INSERT INTO customer (customer_id, first_name, middle_name,
last_name, email, street, city, governorate, national_id, dob,
username, password) VALUES (:customerId, :firstName, :middleName,
:lastName, :email, :street, :city, :governorate, :nationalId,
TO_DATE(:dob, 'YYYY-MM-DD'), :username, :password)'
```

Creates the new customer record. TO_DATE is used to convert the date string from the frontend into Oracle's date format.

```
'INSERT INTO customer_phone (customer_id, customer_phone) VALUES
(:customerId, :phone)'
```

Saves the new customer's phone number, only if one was provided during onboarding.

Staff — Managing Accounts

```
'SELECT dob FROM customer WHERE customer_id = :cid'
```

Fetches a customer's date of birth to verify they are old enough (16+) before staff can open an account for them.

```
'SELECT NVL(MAX(account_number), 10000000000) + 1 AS next_num FROM
account'
```

Generates the next available account number. Starts from 100000000001 if no accounts exist yet.

```
'INSERT INTO account (account_number, account_type, balance, status,
branch_id) VALUES (:accountNumber, :accountType, :balance, 'Active',
:branchId)'
```

Creates a new bank account, defaulting the status to Active.

```
'INSERT INTO customer_account (customer_id, account_number) VALUES
(:customerId, :accountNumber)'
```

Links the newly created account to the customer in the junction table.

```
'UPDATE account SET status = :status WHERE account_number IN (SELECT
account_number FROM customer_account WHERE customer_id = :id)'
```

Freezes or unfreezes all accounts belonging to a specific customer at once.

```
'UPDATE account SET status = :status WHERE account_number =
:accountNumber'
```

Changes the status of a single specific account (e.g. Active or Inactive).

```
'SELECT balance FROM account WHERE account_number = :acc'
```

Checks the account balance before closing it, to prevent closing an account that still holds funds.

```
'UPDATE account SET status = 'Closed' WHERE account_number = :acc'
```

Permanently closes an account after confirming its balance is zero.

Staff — Issuing Cards & Updating Limits

```
'SELECT NVL(MAX(card_id), 5000) + 1 AS next_id FROM card'
```

Calculates the next card ID for a staff-issued card, same logic as customer self-service.

```
'INSERT INTO card (card_id, card_type, card_limit, card_number,
issue_date, expiry_date, cvv, card_status, account_number) VALUES
(:id, :type, :limit, :num, SYSDATE, ADD_MONTHS(SYSDATE, 48), :cvv,
'Active', :accNum)'
```

Issues a card directly and sets it to Active immediately, unlike customer applications which start as Pending.

```
'UPDATE card SET card_limit = :newLimit WHERE card_id = :cardId'
```

Updates the spending limit on a credit card.

Staff — Full Transaction Log

```
'SELECT bt.transaction_id, bt.transaction_type, bt.amount,
bt.transaction_time, bt.status, bt.sender_account_number,
```

```

bt.receiver_account_number, c_sender.first_name || ' ' ||
c_sender.last_name AS sender_name, c_receiver.first_name || ' ' ||
c_receiver.last_name AS receiver_name FROM bank_transaction bt LEFT
JOIN customer_account ca_sender ON bt.sender_account_number =
ca_sender.account_number LEFT JOIN customer c_sender ON
ca_sender.customer_id = c_sender.customer_id LEFT JOIN
customer_account ca_receiver ON bt.receiver_account_number =
ca_receiver.account_number LEFT JOIN customer c_receiver ON
ca_receiver.customer_id = c_receiver.customer_id ORDER BY
bt.transaction_time DESC'

```

Gets every transaction in the system with the full names of both the sender and receiver resolved through two separate joins. Used in the staff transaction log.

Conclusion

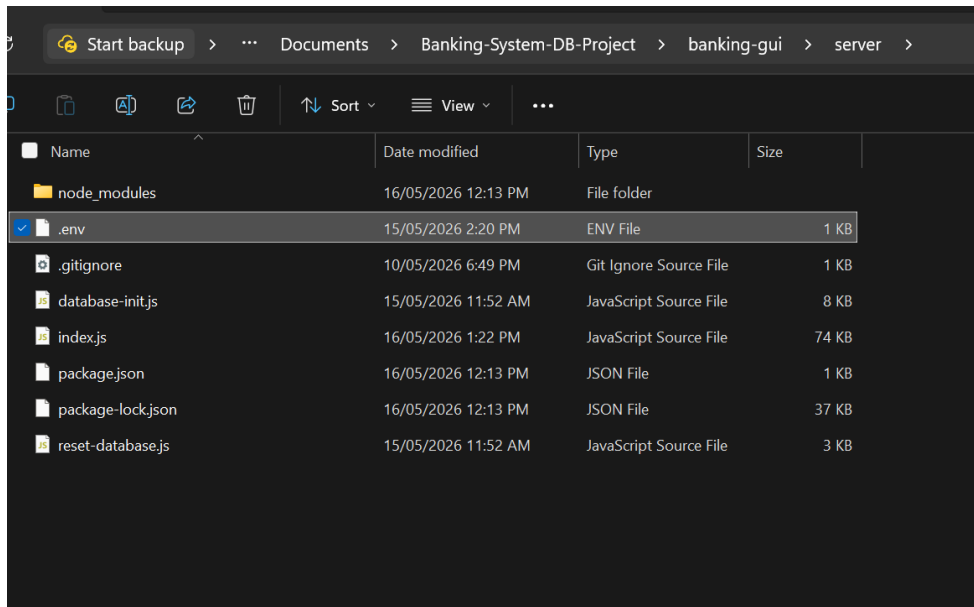
The Banking Database Management System demonstrates the practical implementation of a relational database solution for handling banking operations. The project integrates multiple banking services into a structured and normalized system while ensuring data consistency and efficient operation.

User Manual

To run backend:

***you will need to install nodejs prior**

- 1- Go to project/banking-gui/server
- 2- Right click the folder and select open in terminal
- 3- Run this command “ npm install dotenv”
- 4- In the server folder create a .env file



5- Inside it write:

DB_USER=system

DB_PASSWORD=admin (or your database password)

DB_CONNECTION_STRING=localhost/XEPDB1

6- Back in the terminal write “node index.js”

***you can run ‘ node reset-database.js’ then ‘node index.js’ to reset the database**

7- You should see this:

```
PS C:\Users\btob\OneDrive\Documents\Banking-System-DB-Project\banking-gui\server> node index.js
◇ injected env (3) from .env // tip: ✎ enable debugging { debug: true }
◇ injected env (0) from .env // tip: ✎ enable debugging { debug: true }

🔍 Checking database schema...

📊 Database Status:
Found 0/14 required tables
X Missing: CUSTOMER, BRANCH, DEPARTMENT, JOBS, EMPLOYEES, ACCOUNT, LOAN, CARD, BANK_TRANSACTION, DEPENDANTS, CUSTOMER_PHONE, EMPLOYEE_PHONE, CUSTOMER_ACCOUNT, DEPARTMENT_MANAGERS

🔧 Running full initialization...

📦 Creating 14 schema objects...

✓ [1/14] Created Customer
✓ [2/14] Created Branch
✓ [3/14] Created Department
✓ [4/14] Created Jobs
✓ [5/14] Created Employees
✓ [6/14] Created Account
✓ [7/14] Created Loan
✓ [8/14] Created Card
✓ [9/14] Created Bank_Transaction
✓ [10/14] Created Dependants
✓ [11/14] Created Customer_Phone
✓ [12/14] Created Employee_Phone
✓ [13/14] Created Customer_Account
✓ [14/14] Created Department_Managers
```

To run front-end:

1- Open project/banking-gui/client

2- Right click and open terminal

3- Write the following commands:

“npm install”

“npm run dev”

4- If you get this error:

```
PS C:\Users\btob-\OneDrive\Documents\Banking-System-DB-Project\banking-gui\client> npm run dev
npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running scripts is disabled on this system. For
more information, see about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ npm run dev
+ ~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
```

write this command “Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process”,
then write “npm run dev”

5- You should see this:

```
PS C:\Users\btob-\OneDrive\Documents\Banking-System-DB-Project\banking-gui\client> Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process
PS C:\Users\btob-\OneDrive\Documents\Banking-System-DB-Project\banking-gui\client> npm run dev

> banking-gui@0.0.0 dev
> vite

VITE v8.0.11 ready in 1014 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Open the website and the app is ready!